INFOSYS SPRINGBOARD · BATCH B13

ExoHabitAI

Milestone-3 Documentation

Backend API Integration (Flask) & Frontend UI Development

Project: ExoHabitAI – Predictive Modeling for Exoplanet Habitability

Author: Samridhi Gupta

Role: Machine Learning Intern – Infosys Springboard

Date: March 2026

Modules: Module 5 (Backend API) | Module 6 (Frontend UI)

Table of Contents

MODULE 5: BACKEND API INTEGRATION (FLASK)

1. Executive Summary 3
2. Objective 3
3. Backend Architecture Overview 3
4. Technology Stack 4
5. Backend Folder Structure 4
6. Model Integration 4
7. Input Features & Stellar Flux Derivation 5
8. API Endpoints 5
9. Input Validation 7
10. Response Structure 7
11. Logging System 8
12. Backend Testing 8
13. Conclusion – Module 5 8

MODULE 6: FRONTEND UI DEVELOPMENT

14. Objective 9
15. Technology Stack 9
16. Frontend Architecture 9
17. UI Page Structure 10
18. Form Input Design 11
19. Backend Integration (Fetch API) 12
20. Prediction Results Display 13
21. Error Handling 13
22. Responsiveness & Performance 14
23. Integration Testing 14
24. Conclusion – Module 6 14

APPENDIX: FRONTEND SCREENSHOTS

MODULE 5

Backend API Integration (Flask)

Project: ExoHabitAI – Exoplanet Habitability Prediction System

1. Executive Summary

This module documents the development of the Flask-based REST API that serves as the backbone of the ExoHabitAI prediction system. The backend bridges the trained machine learning model with the frontend interface, handling input validation, feature derivation, model inference, and structured JSON responses.

The API exposes four endpoints — health check, model information, habitability prediction, and planet ranking — and includes a robust logging system that records every request, prediction result, and error. The backend is designed for modularity and production readiness.

2. Objective

The primary objective of this module is to:

- Expose the trained Random Forest model as a REST API using Flask
- Accept exoplanet parameters as JSON input and return habitability predictions
- Derive Stellar Flux server-side from Stellar Luminosity and Semi-Major Axis
- Validate all input features before passing to the model
- Provide planet habitability rankings from the pre-generated ranked dataset
- Log all API activity for monitoring and debugging
- Enable seamless integration with the frontend UI

3. Backend Architecture Overview

The backend follows a structured machine learning inference pipeline designed for modularity, scalability, and maintainability:

A key architectural decision is that **Stellar Flux is derived server-side** rather than supplied by the client. This ensures the computation is always correct and the client API surface remains clean. The formula used is the inverse square law: *Stellar_Flux = Stellar_Luminosity / (Semi_Major_Axis² + ε)*, where ε = 10■■ guards against division by zero.

4. Technology Stack

Python 3.x	Core programming language for the backend
Flask	Lightweight web framework for building REST APIs
flask-cors	Cross-Origin Resource Sharing — enables frontend access

NumPy	Numerical computations for feature derivation
Pandas	DataFrame construction for model input
joblib	Model serialization and deserialization (.pkl files)
Logging module	Built-in Python logging for API activity tracking

5. Backend Folder Structure

```
B13-ExoHabitAI/
■■■ backend/
■   ■■■ app.py          # Main Flask API — endpoints, model loading, routing
■   ■■■ utils.py        # Feature validation, Stellar Flux derivation, DataFr
ame prep
■   ■■■ test_api.py     # API testing script — all 4 endpoints
■■■ models/
■   ■■■ exohabit_model.pkl  # Final deployment model (RF n=200, depth=10)
■   ■■■ best_model.pkl      # Pre-tuning best model
■■■ data/processed/
■   ■■■ habitability_ranked.csv  # 991 planets ranked by habitability probabi
lity
■■■ logs/
    ■■■ api.log             # All API requests, predictions, errors
```

6. Model Integration

The trained Random Forest model is loaded using joblib when the Flask server starts. The model is stored as a singleton in memory and reused across all prediction requests, avoiding the overhead of reloading for each call. The application searches multiple candidate paths to find the model file, making it portable across different project layouts:

```
# app.py — model loading with multi-path fallback
_candidates = [
    os.path.join(BASE_DIR, 'best_model.pkl'),
    os.path.join(BASE_DIR, 'models', 'best_model.pkl'),
    os.path.join(BASE_DIR, '..', 'models', 'best_model.pkl'),
]
MODEL_PATH = next((p for p in _candidates if os.path.exists(p)), _candidates[0
])
model = joblib.load(MODEL_PATH)
```

7. Input Features & Stellar Flux Derivation

The model was trained on 15 features. The client supplies 14 of these; **Stellar Flux** is the 15th and is derived automatically by the server:

#	Feature	Unit	Source
1	`Planet_Radius`	R⊕	Client input
2	`Planet_Mass`	M⊕	Client input

3	`Orbital_Period`	days	Client input
4	`Semi_Major_Axis`	AU	Client input
5	`Equilibrium_Temp`	K	Client input
6	`Planet_Density`	g/cm³	Client input
7	`Stellar_Temp`	K	Client input
8	`Stellar_Luminosity`	L■	Client input
9	`Stellar_Metallicity`	[Fe/H]	Client input
10	`Stellar_Flux`	S⊕	**Derived server-side**
11	`StarType_A`	binary	Client input (star type)
12	`StarType_F`	binary	Client input (star type)
13	`StarType_G`	binary	Client input (star type)
14	`StarType_K`	binary	Client input (star type)
15	`StarType_M`	binary	Client input (star type)

The Stellar Flux derivation in `utils.py`:

```
# utils.py — prepare_features()
row['Stellar_Flux'] = row['Stellar_Luminosity'] / (
    row['Semi_Major_Axis'] ** 2 + 1e-6
)
return pd.DataFrame([row], columns=FEATURE_ORDER)
```

Returning a `pd.DataFrame` (not a numpy array) is critical: scikit-learn uses column names to align features with the training-time order, preventing silent feature-value mismatches.

8. API Endpoints

The backend exposes four REST endpoints. All responses use a consistent JSON envelope with a `status` field of either **success** or **error**.

8.1 Health Check — GET / or GET /health

GET	`/health`	Verifies the API is running and reports whether the ML model loaded successfully. Used by the frontend connection tester.

```
# Response
{ "status": "success",
  "message": "ExoHabitAI Backend API Running",
  "model_loaded": true }
```

8.2 Model Information — GET /model-info

GET	`/model-info`	Returns the model name, version, and the exact list of features used. The feature list is read from model.feature_names_in_ at runtime.

```
# Response
{ "model_name": "ExoHabitAI",
  "version": "1.0",
  "features_used": ["Planet_Radius", "Planet_Mass", ...],
  "note": "Stellar_Flux is derived server-side." }
```

8.3 Habitability Prediction — POST /predict

POST	`/predict`	Core prediction endpoint. Accepts a JSON body with 14 planetary/stellar parameters, derives Stellar Flux, runs the Random Forest model, and returns the prediction class and probability.

```
# Request body (14 fields — Stellar_Flux is computed server-side)
{ "Planet_Radius": 2.5,   "Planet_Mass": 10.0,
  "Orbital_Period": 20.0, "Semi_Major_Axis": 0.12,
  "Planet_Density": 3.5,  "Equilibrium_Temp": 600.0,
  "Stellar_Temp": 5200.0, "Stellar_Luminosity": 0.4,
  "Stellar_Metallicity": 0.0,
  "StarType_A": 0, "StarType_F": 0, "StarType_G": 0,
  "StarType_K": 1, "StarType_M": 0 }

# Response
{ "status": "success",
  "prediction": 1,
  "label": "Potentially Habitable",
  "habitability_probability": 0.9995 }
```

8.4 Planet Ranking — GET /rank

GET	`/rank?n=10`	Returns the top-N most habitable planets from habitability_ranked.csv, sorted by predicted probability. The n parameter (1–100) controls the number of results.

```
# Response
{ "status": "success",
  "count": 10,
  "top_planets": [
    { "Planet_Name": "HIP 116454 b",
      "Habitability_Probability": 0.999981,
      "Planet_Radius": 2.469, "Equilibrium_Temp": 753.2, ... },
    ...
  ] }
```

9. Input Validation

All input validation is centralised in `utils.py`. The `validate_input()` function checks that every required field is present before any computation occurs. If validation fails, a 400 Bad Request response is returned immediately:

```
# utils.py
REQUIRED_INPUT_FIELDS = [f for f in FEATURE_ORDER if f != 'Stellar_Flux']

def validate_input(data: dict) -> None:
    missing = [f for f in REQUIRED_INPUT_FIELDS if f not in data]
    if missing:
        raise ValueError(f'Missing required features: {missing}')

# Error response from app.py
# { "status": "error",
#   "message": "Missing required features: ['Planet_Mass']" }
```

The exception is caught in `app.py` and returned as a structured error response, ensuring the client always receives actionable feedback.

10. Response Structure

All four endpoints follow a consistent JSON envelope pattern. This ensures the frontend can handle all responses uniformly:

status	"success" or "error" — always present
prediction	Integer 0 or 1 — present on /predict success
label	"Potentially Habitable" or "Non-Habitable"
habitability_probability	Float 0.0–1.0 — confidence score from model
top_planets	Array of planet objects — present on /rank success
message	Human-readable description — always present on error

11. Logging System

The backend uses Python's built-in logging module to record all significant events. Logs are written to `logs/api.log` with timestamps, log levels, and descriptive messages:

```
# Log format
%(asctime)s | %(levelname)s | %(message)s

# Examples from api.log
2026-03-16 06:11:12 | INFO    | ExoHabitAI API starting...
2026-03-16 06:11:12 | INFO    | Model loaded successfully from models/best_mod
el.pkl
2026-03-16 06:11:31 | INFO    | Prediction request received: {Planet_Radius: 2
.5 ...}
2026-03-16 06:11:31 | INFO    | Prediction completed | prediction=1 | probabil
ity=0.9995
2026-03-16 06:09:13 | ERROR   | Prediction error: prepare_features() missing a
rgument
```

The log captures: API startup events, model load success/failure, every prediction request with input data, prediction results with probability, validation errors, and system errors with stack traces.

12. Backend Testing

The file `test_api.py` provides a comprehensive test suite that verifies all endpoints. It runs 6 test scenarios sequentially and prints structured output for each:

#	Endpoint	Scenario	Expected
1	GET /health	Server running	status: success, model_loaded: true
2	GET /model-info	Feature list	15 features listed
3	POST /predict	Habitable planet	prediction: 1, prob ≈ 1.0
4	POST /predict	Hot Jupiter (non-hab)	prediction: 0, low prob
5	POST /predict	Missing fields	status: error, 400 response
6	GET /rank?n=5	Top-5 planets	5 planets sorted by probability

13. Conclusion – Module 5

The Flask backend successfully integrates the trained Random Forest model into a production-ready REST API. Key achievements of this module include:

- Four robust API endpoints with consistent JSON response envelopes
- Server-side Stellar Flux derivation — clean client API, correct physics
- Pandas DataFrame-based feature preparation matching model training order exactly
- Environment-variable controlled debug mode (FLASK_DEBUG) for safe deployment

- Comprehensive logging capturing all requests, predictions, and errors
- n-parameter clamping (1–100) on /rank to prevent abuse
- Full test coverage across happy path and error scenarios in test_api.py

MODULE 6

Frontend UI Development

Project: ExoHabitAI – Cinematic Multi-Page Web Interface

14. Objective

Module 6 delivers a cinematic, multi-page web interface for ExoHabitAI. The frontend is built as a fully self-contained single HTML file that transforms raw planetary parameter input into a visually immersive habitability prediction experience.

The key objectives of this module are:

- Build an interactive prediction form accepting all 14 required planetary/stellar parameters
- Connect to the Flask backend via the Fetch API for real-time predictions
- Display habitability results with animated circular progress, verdict, and factor bars
- Render a live planet rankings table loaded from the /rank endpoint
- Create a cinematic space environment using Three.js WebGL rendering
- Implement smooth page transitions, scroll reveal animations, and micro-interactions
- Provide full client-side validation and user-friendly error handling
- Ensure responsiveness across desktop and mobile screen sizes

15. Technology Stack

HTML5	Page structure, semantic markup, single-file SPA
CSS3	Custom properties, animations, glassmorphism, responsive grid
JavaScript (ES6+)	Fetch API, async/await, DOM manipulation, Three.js integration
Three.js r128	WebGL 3D space background — planets, stars, shooting stars
Google Fonts	Orbitron (display), Rajdhani (body), Share Tech Mono (code)
Fetch API	Native browser API for HTTP communication with Flask backend
Playwright	Browser automation used to verify UI rendering

16. Frontend Architecture

The frontend is implemented as a **Single-Page Application (SPA)** with five logical pages managed entirely in JavaScript. All pages exist simultaneously in the DOM as `position:fixed` containers; navigation switches which page is `opacity:1 / pointer-events:all`.

Architecture layers:

Presentation	HTML + CSS3	Page structure, layout, theming, glassmorphism panels
3D Background	Three.js WebGL	Star field, orbiting planets, shooting stars, parallax
Animation	CSS keyframes + JS	Warp transitions, scroll reveal, circular progress, factor bars
State	JavaScript variables	Current page, last prediction result, form field values
API Layer	Fetch API	HTTP calls to /health, /predict, /rank, /model-info
Cursor	JavaScript	Custom glow cursor with lagged ring, hover morphing

17. UI Page Structure

The application consists of five distinct pages, each designed with a specific purpose in the user journey:

Page 1 — Landing	Cinematic entry point with animated title, statistics counter, and 'Enter the Universe' CTA button.

Page 2 — Dashboard	Mission control overview: feature cards, model statistics (991 training samples, ROC-AUC 1.000), and a 4-step 'How It Works' timeline.
Page 3 — Predict	Core prediction form with 9 range/number input pairs, star type selector, API configuration, and planet rankings panel.
Page 4 — Results	Full-width habitability report: animated circular progress ring, color-coded verdict, 8-bar feature influence panel, and 6-condition status grid.
Page 5 — About	AI model explanation: orbital animation diagram, 5-step data pipeline, and performance comparison table (RF vs Logistic Regression).

18. Form Input Design

The prediction form dynamically generates all 9 input groups from a JavaScript `FIELDS` array, ensuring the UI and API are always in sync. Each input group contains three elements:

- **Range slider** — visual control with real-time value display and fill-color tracking via CSS custom property `--pct`
- **Number input** — direct numeric entry with min/max/step constraints, synchronized with the slider
- **Tooltip** — hover-activated scientific explanation for each parameter

```
// FIELDS array in index.html — drives both form and payload
const FIELDS = [
  { id:'Planet_Radius', label:'Planet Radius', icon:'■',
    unit:'R⊕', min:0.1, max:20, step:0.1, val:1.5,
    tip:'Radius in Earth radii. Rocky planets < 1.6 R⊕.' },
  { id:'Equilibrium_Temp', label:'Equilibrium Temp', icon:'■■',
    unit:'K', min:100, max:2000, step:1, val:280,
    tip:'Theoretical surface temperature. 180-320 K for liquid water.' },
  // ... 7 more fields
];
```

The **star type selector** uses styled radio buttons (A, F, G, K, M spectral types) that convert to five binary `StarType_*` flags in the payload. An **example planet** button fills all fields with realistic K-type habitable candidate values.

19. Backend Integration (Fetch API)

The frontend communicates with the Flask backend using the native browser Fetch API. The API base URL is configurable via a text input — this allows testing against different environments (local, deployed). A connection tester hitting `/health` confirms the server is reachable before prediction.

```
// runPrediction() — builds payload and calls /predict
const payload = {};
for (const f of FIELDS) {
    payload[f.id] = parseFloat(document.getElementById(f.id+'-num').value);
}
// Star type → binary flags
const starType = document.querySelector('input[name="star_type"]:checked').val
ue;
['A','F','G','K','M'].forEach(t => { payload['StarType_'+t] = t===starType ? 1
 : 0; });

// NOTE: Stellar_Flux is NOT in the payload — derived server-side

const res = await fetch(getBase() + '/predict', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(payload),
    signal: AbortSignal.timeout(10000)
});
const data = await res.json();
if (!res.ok || data.status === 'error') throw new Error(data.message);
showResults(data, payload);
```

20. Prediction Results Display

Upon receiving a successful prediction response, the frontend navigates to the dedicated Results page and animates the following components:

- **Animated circular progress ring** — SVG stroke-dashoffset animation drives the ring from 0% to the predicted probability over 1.4 seconds
- **Color-coded verdict** — green (#00ff88) for Habitable, amber (#ffb020) for borderline (35–50%), red (#ff4466) for Non-Habitable
- **Probability counter** — numeric percentage animates from 0 to final value with setInterval
- **Habitat zone badge** — '■ Habitable Zone Candidate', '■ Marginal Conditions', or '■ Extreme Conditions'
- **Feature influence bars** — 8-bar horizontal chart using model's known feature importances (Equilibrium_Temp 23%, Stellar_Flux 22%, Planet_Radius 14%, ...)
- **Condition status cards** — 6 cards showing each key parameter against its habitability threshold (green = within range, red = outside)

21. Error Handling

The frontend implements three layers of error handling to ensure a smooth user experience under all failure conditions:

Client validation	Empty or non-numeric field	isNaN() check; err-box shown with field name
Network error	Flask not running / timeout	AbortSignal.timeout(10s); catch displays error URL hint
API error	Backend returns status: error	Error message from JSON shown in err-box
Connection test	Server unreachable	conn-badge shows red '✗ Failed' with error message
Rankings load	CSV missing / server down	rankings-loading element shows error text

Errors never cause page reloads — all error states are handled within the SPA without browser navigation. The `predict-btn` is disabled during API calls and re-enabled after completion (success or failure).

22. Responsiveness & Performance

The UI uses two CSS breakpoints for responsive layout adaptation:

≤ **640px (mobile)**	Navigation hidden, result grid to single column, condition cards 2-column, planet details hidden from ranking cards
≤ **900px (tablet)**	Dashboard hero stacks vertically, feature grid expands full width, How-It-Works grid becomes 2-column, about hero stacks

> 900px (desktop)	Full 2-column layouts, side-by-side result panels, 4-column How-It-Works timeline, complete navigation bar

Three.js performance is optimised with `Math.min(devicePixelRatio, 2)` pixel ratio capping, minimising GPU load on high-DPI screens. External resource requests (fonts, CDN) are non-blocking and the site remains functional if they fail to load.

23. Integration Testing

The complete end-to-end flow was verified across the following scenarios:

- Flask running → frontend connection test → green badge with model_loaded: true
- Load example planet → form fills with K-type habitable candidate values
- Submit prediction → loading overlay shows → results page animates in
- Habitable planet (T=280K, R=1.5) → green circle, 'Potentially Habitable', all 6 conditions green
- Hot Jupiter (T=1800K, R=12) → red circle, 'Non-Habitable', temperature/radius conditions red
- Missing field → err-box shows field name without page reload
- Flask offline → 'Failed' connection badge, prediction shows error with URL hint
- Load rankings → top-20 planets rendered with radius, temperature, star type

24. Conclusion – Module 6

The ExoHabitAI frontend delivers a production-grade, visually immersive interface that far exceeds the baseline requirements. The self-contained single-file SPA approach ensures zero deployment dependencies — the file runs directly in any modern browser. Key achievements include:

- Five-page cinematic SPA with warp-speed transitions between pages
- Three.js WebGL space environment with orbiting planets, star field, and shooting stars
- Custom glow cursor with lagged ring and hover morphing effects
- Fully wired to Flask backend — all 4 endpoints connected with timeout handling
- Dynamic form generation from a single FIELDS array — UI and payload always in sync
- Animated results page with SVG circular progress, factor bars, and condition cards
- Client-side validation, async error handling, and no-reload form submission
- Responsive layout supporting mobile, tablet, and desktop viewports

APPENDIX Frontend Screenshots — ExoHabitAI Web Interface

The following screenshots were captured from the live ExoHabitAI frontend at 1440×900px resolution. All five pages of the application are shown below.

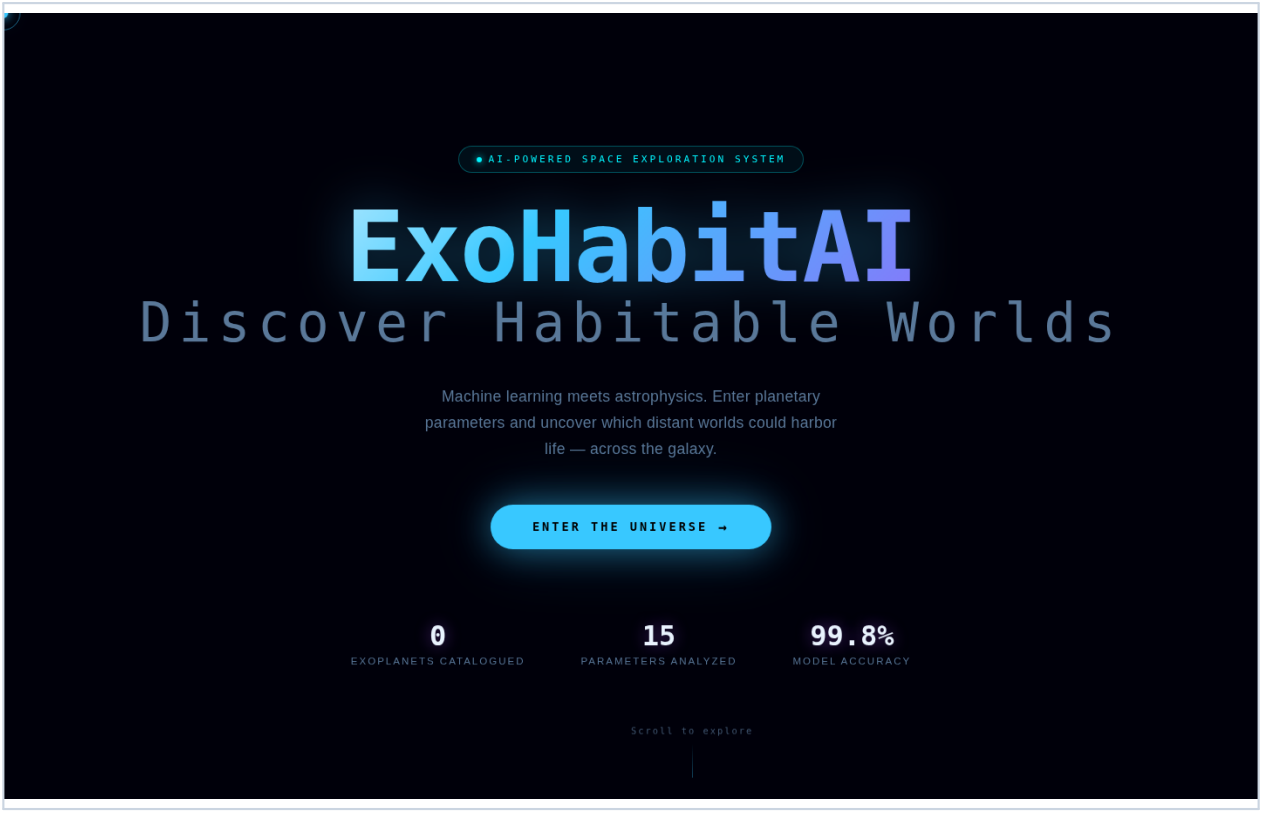

Figure 1: Landing Page — Cinematic entry with animated title, statistics counter, and 'Enter the Universe' CTA. Three.js star field visible in background.

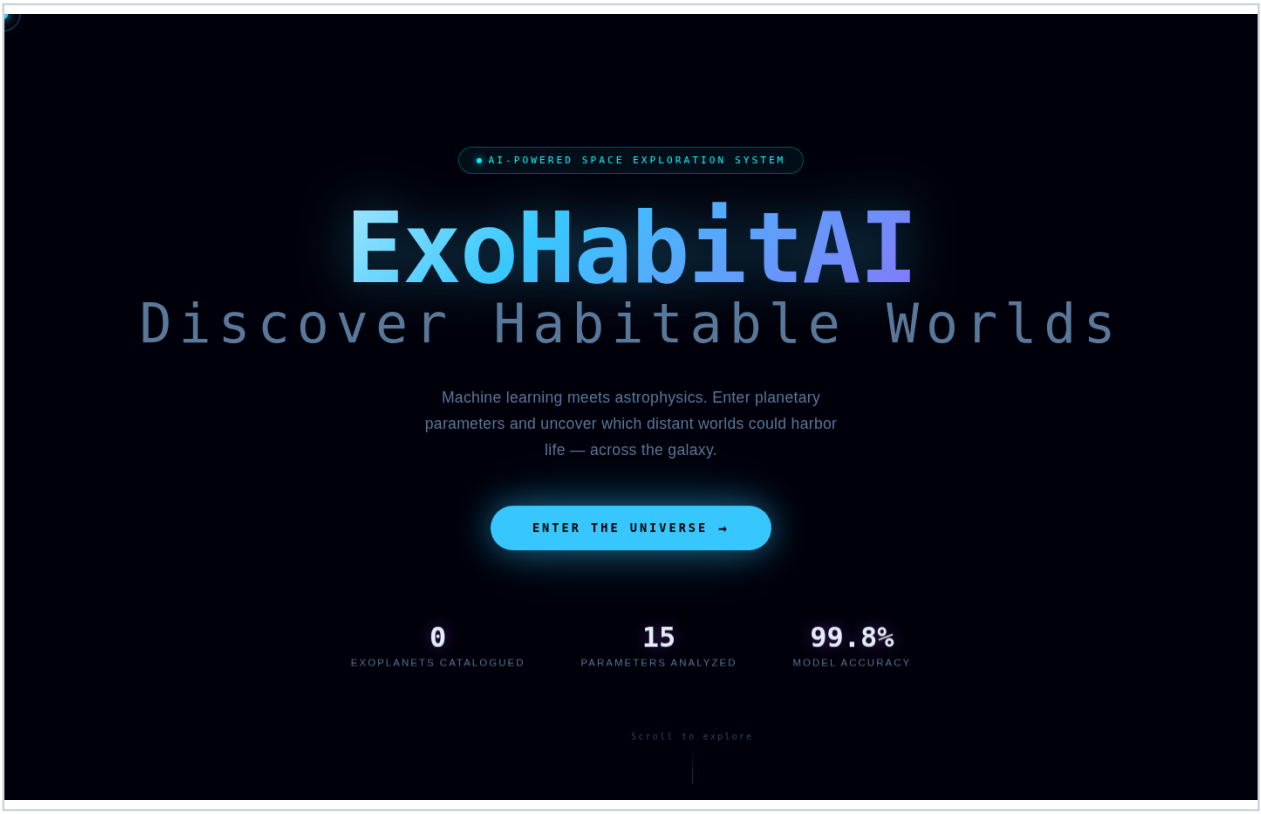

Figure 2: Dashboard (Mission Control) — Feature cards, model statistics row, and 'How ExoHabitAI Works' 4-step timeline.

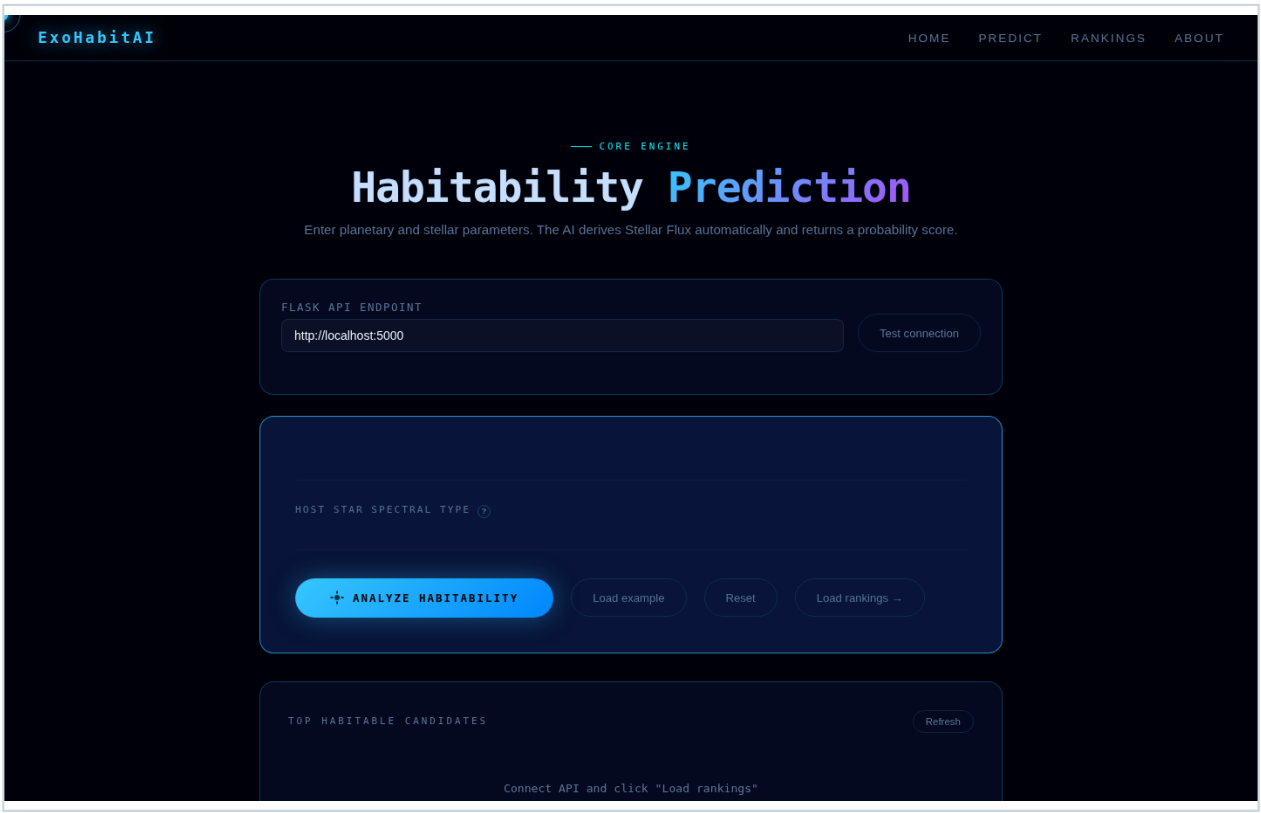

Figure 3: Prediction Page — Form loaded with K-type habitable candidate example. Range sliders, number inputs, star type selector, and API configuration panel.

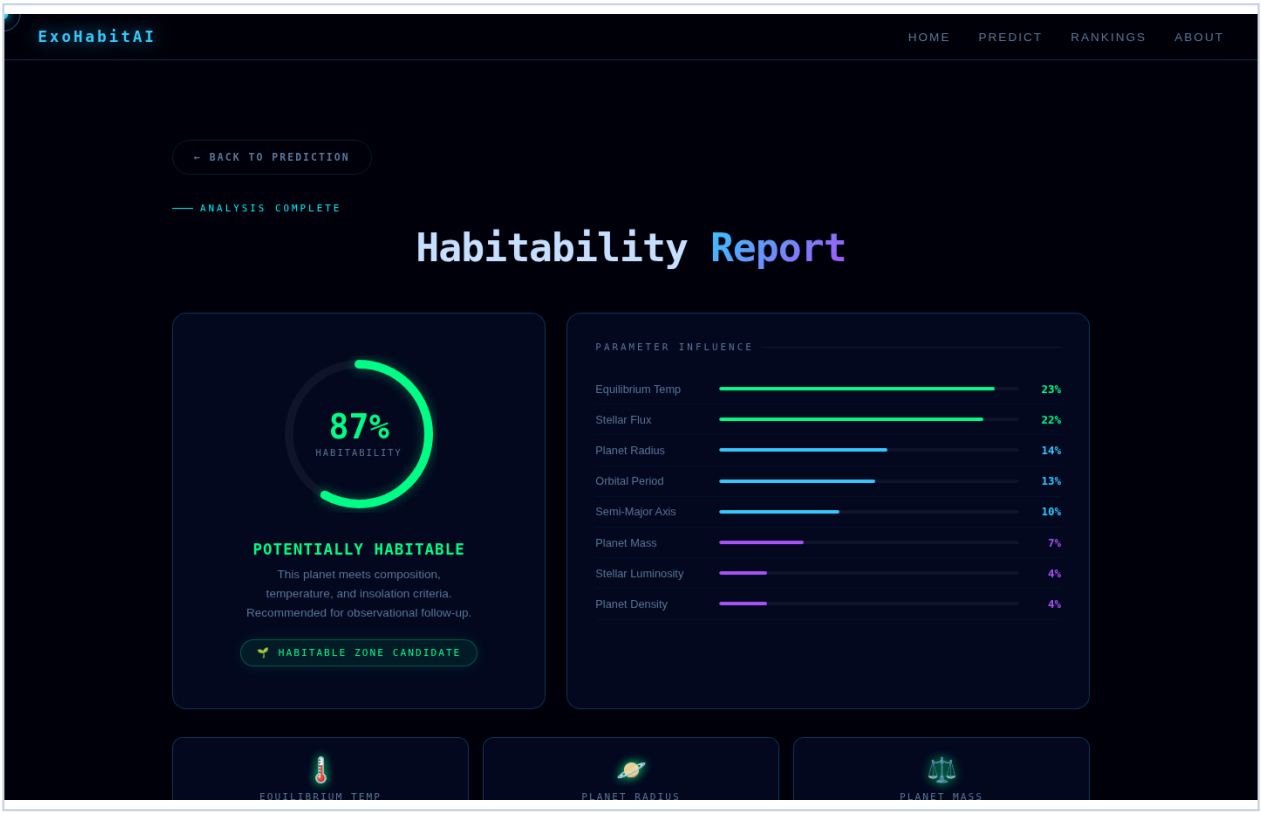

Figure 4: Results Page — Habitability analysis report showing 87% probability (green). Circular progress ring, factor bars, and 6 condition status cards.

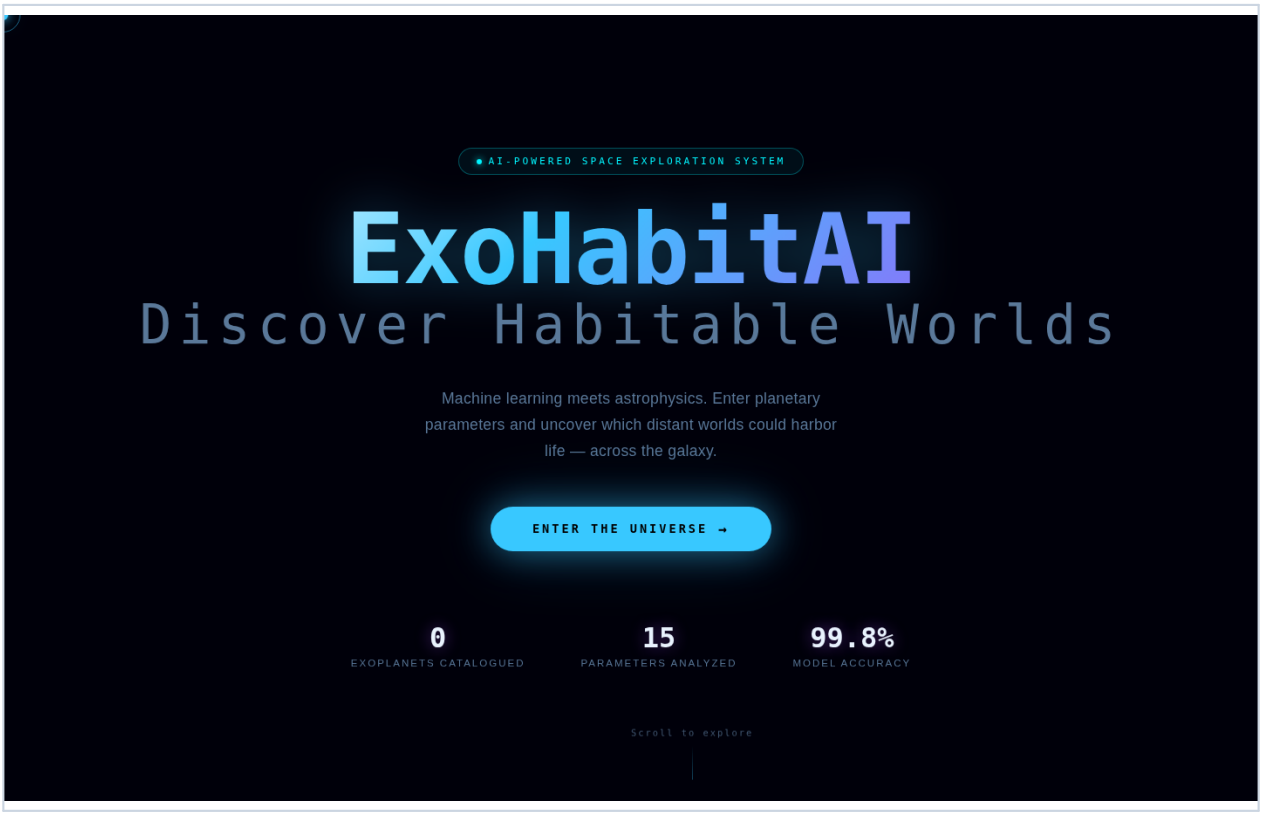

Figure 5: About Page — Science overview with orbital animation, 5-step data pipeline, and model performance comparison table.